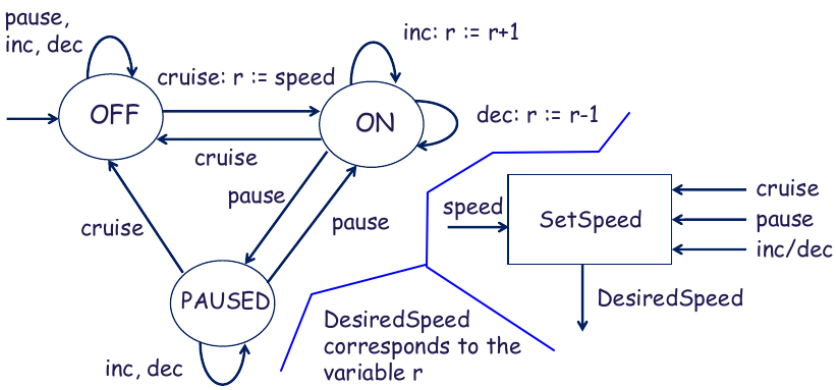
Suman Sourav
Jonathan Munive

AALBORG UNIVERSITY
DENMARK

Models and Tools for Cyber-Physical Systems
Exercise sheet 2
WITH SOLUTIONS

Exercise 1: UPPAAL simplified cruise controller
Consider the cruise controller from the lecture. One sub-component is the `SetSpeed` component (Chapter 1, Slide 23). This component takes as input the current `speed` from the car and several signals (`cruise`, `pause`, `inc`, `dec`) from the driver.



Solve the following tasks; the task marked with (∗) is optional.

(a) Download the initial UPPAAL model from Moodle.

(b) Check whether the desired speed is between 40 and 80 speed units. You can use the following Timed Computation Tree Logic (TCTL) property φ:

$$\forall\Box\ \texttt{desired_speed} \geq 40 \wedge \texttt{desired_speed} \leq 80$$

(UPPAAL syntax: `A[] (desired_speed >= 40 && desired_speed <= 80)`)

(c) Modify the component `SetSpeed` such that the formula φ (above) is satisfied.

(Side note: The property φ is not a realistic requirement of a cruise controller.)

(∗) Create a component `DriverFast` that increases the variable `desired_speed` until it reaches the value 78. When the variable `desired_speed` is at value 78, the component enters a location `stable`. Do not modify the other components. Verify the desired behavior by checking the following TCTL property:

$$\forall\Diamond\ \texttt{DriverFast.stable} \wedge \texttt{desired_speed} = 78$$

(UPPAAL syntax: `A<> DriverFast.stable && desired_speed == 78`)

Hints: Remember to replace the old `Driver` component with the new `DriverFast` component in the system declarations. If your driver exceeds the speed, try to model a maximum speed of 77 instead and still check for a maximum speed of 78; read the explanation in the solution to understand why this may help.

.....Solution sketch .....

Download the UPPAAL model solution from Moodle.

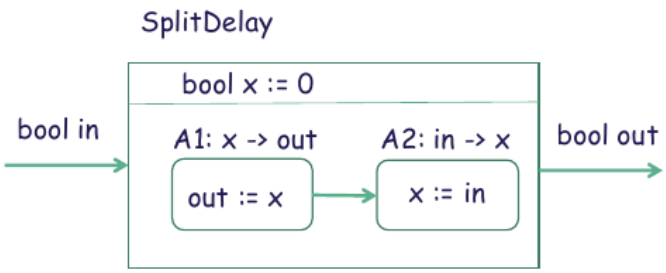
The comparison of `desired_speed` to 77 in the guards may be counterintuitive. You can use the simulator (best initialize the speed to 76 in the global "Declarations" to shorten the process) to see that in the round where the `DriverFast` component issues the `pause`, the `SetSpeed` component still takes the `inc` transition. The reason is that transitions are selected before any updates are applied, and hence the `pause` only takes effect in the next round. This shows that it is important to understand the semantics of the model.

Exercise 2: Parallel composition

Consider the component `DoubleSplitDelay` and its textual description

$$(\texttt{SplitDelay}[\texttt{out} \mapsto \texttt{temp}] \parallel \texttt{SplitDelay}[\texttt{in} \mapsto \texttt{temp}]) \setminus \texttt{temp}.$$

This component is similar to the component `DoubleDelay`, except that we use instances of the component `SplitDelay` (Chapter 2, Slide 54) instead of `Delay`.

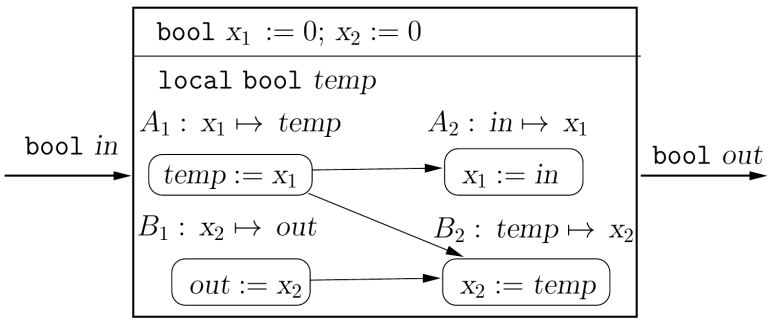


(a) Give the block diagram of `DoubleSplitDelay` (which consists of (state, input, output, and local) variables, initialization, tasks, and precedence constraints).

(b) What are the await dependencies for `DoubleSplitDelay` (among output and input variables)?

.....Solution sketch .....

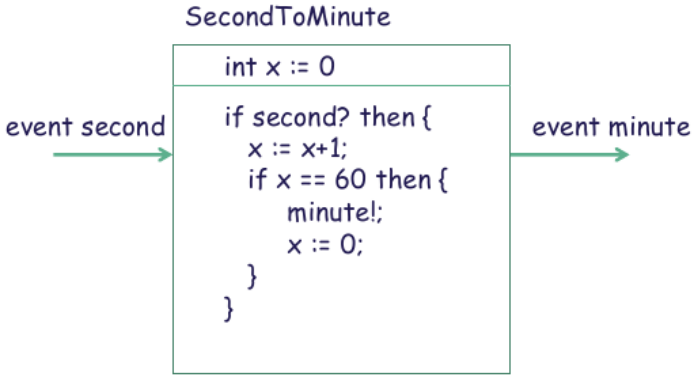
(a) The component `DoubleSplitDelay` has input variable `in`, output variable `out`, state variables `x1` and `x2`, and local variable `temp`, all of type bool. Its reaction description consists of 4 tasks as shown below (the task names are arbitrary).



(b) There are no await dependencies (output variable "`out`" does not await input variable "`in`").

Exercise 3: Second to hour

Recall the event-triggered component `SecondToMinute` (Chapter 2, Slide 78) with the input event variable `second` and the output event variable `minute` such that `minute` is present every 60th time the event `second` is present.



Now suppose we want to design an event-triggered component `SecondToHour` with an input event variable `second` and an output event variable `hour`, such that the output event hour is present every 3600th time the event `second` is present.

Give a textual description (see Exercise 2) of the component `SecondToHour` from the component `SecondToMinute` using the operations of parallel composition, instantiation, and output hiding.

.....Solution sketch .....

The only intricate aspect is the substitution. We first have to rename `minute` to `hour` before we can rename `second` to `minute`.

$$(\texttt{SecondToMinute} \parallel \texttt{SecondToMinute}[\texttt{minute} \mapsto \texttt{hour}][\texttt{second} \mapsto \texttt{minute}]) \setminus \texttt{minute}.$$